OmniXtra UI/UX Product Requirements & Business Requirements Document

1. Product Overview

OmniXtra is an AI-native, multi-tenant Learning Management System for small-to-medium businesses and startups. It helps teams launch internal training, customer education, partner enablement, onboarding, and compliance training without a sales-led implementation cycle.

The product differentiator is a learner-facing AI tutor grounded in each tenant's course content through retrieval-augmented generation. The tutor should feel useful, trustworthy, and clearly advisory.

Self-serve onboarding is central to the product. A new customer should be able to sign up, create a tenant workspace, configure basic branding, create or upload course content, invite learners, and track progress with minimal support.

2. Business Goals

Goal	UX implication
First 10 paying tenants by end of month 4	Signup, onboarding, billing, and first-course creation must feel polished enough for early paid customers.
$10k MRR by end of month 9	Plan value, limits, upgrade prompts, and progress visibility must be easy to understand.
Signup to first published course under 30 minutes at P50	Onboarding and course creation must be direct, guided, and low-friction.
Learner course completion above 65%	Learner flows must make progress, next steps, resume, quiz feedback, and completion obvious.
AI tutor weekly active usage above 40% among enrolled learners	The tutor must be discoverable inside the course context without disrupting learning.
Tenant 30-day retention above 80%	Admins must quickly see value through completion reporting, learner progress, course status, and usage.

Design should make the product feel self-explanatory. Admin workflows should be simple, learner experiences should reduce friction, AI should be visible but not intrusive, and dashboards should help customers see early value.

3. Target Personas

Pia, Head of People

Field	Summary
Context	Leads people operations at a 20-200 person company without a dedicated L&D team.
Primary job	Launch onboarding and compliance training quickly.
Main pain points	Too many manual documents, repeated onboarding questions, unclear completion tracking.

Field	Summary
UX needs	Guided setup, course templates or clear blank-start flows, simple learner invitations, visible completion status.
Success signals	First course launched within a week; required learners are enrolled and progressing.

Marco, Customer Success Lead

Field	Summary
Context	Runs customer education for a growing B2B SaaS company.
Primary job	Educate customers and reduce onboarding/support tickets.
Main pain points	Customers miss setup steps, support repeats the same answers, enablement material is scattered.
UX needs	Easy course organization, shareable learner experience, AI tutor that answers from course content, reporting by course.
Success signals	Fewer onboarding tickets and better customer activation.

Sofia, Course Creator

Field	Summary
Context	Subject-matter expert or instructor creating course material.
Primary job	Build structured lessons with text, video, images, and quizzes.
Main pain points	Authoring tools are too complex, publishing rules are unclear, media workflows are slow.
UX needs	Clear course hierarchy, block editor, media upload states, preview mode, draft/publish clarity.
Success signals	Course built and published quickly without engineering or support help.

Leo, Learner

Field	Summary
Context	Employee, customer, or partner assigned training.
Primary job	Complete assigned learning with minimal confusion.
Main pain points	Unclear next lesson, lost progress, quiz anxiety, needing help while learning.
UX needs	Resume CTA, progress bar, focused player, accessible mobile layout, AI tutor inside course context.
Success signals	Course completed, quiz passed, certificate downloaded where applicable.

Ada, Tenant Admin

Field	Summary
Context	IT, operations, or admin owner for a tenant workspace.
Primary job	Manage users, fixed-role assignment, branding, billing, reports, and tenant settings.
Main pain points	Needing support for routine configuration, unclear plan limits, weak reporting.

Field	Summary
UX needs	Operational admin shell, clear settings, role-aware navigation, billing status, CSV export, confirmations for sensitive changes.
Success signals	Tenant runs training workflows without support tickets.

---

4. MVP Scope For UI/UX

MVP Includes

- Signup and tenant creation.
- Login and magic-link flow.
- Tenant onboarding wizard.
- Fixed role assignment for Tenant Admin, Course Owner, and Learner.
- Admin dashboard.
- Tenant branding: logo, primary colour, custom subdomain.
- Course list.
- Course builder.
- Lesson editor.
- Block-based lesson content: text, video, image, quiz.
- Draft/publish workflow.
- Media upload flow.
- Learner course catalog and enrolments.
- Course player.
- Lesson progress and resume.
- Quiz flow.
- Completion state.
- Certificate download.
- AI tutor chat panel in course context.
- Learner progress report.
- CSV export.
- Billing and plan page.
- Transactional email confirmation states where the UI needs to explain what happened.

Explicitly Excluded From MVP

- SCORM/xAPI import.
- SSO, MFA, and custom roles.
- Discussion forums.
- Live classes.
- Marketplace.
- Custom domains.
- Native mobile apps.
- Deep analytics and custom dashboards.
- MCP or agentic AI tools.

- AI course generation.
- Voice tutor.
- Gamification and social learning.

Future-phase concepts can be noted as roadmap items, but they should not appear in MVP designs as available functionality.

---

5. Product Principles For Design

- Self-serve first: the user should not need documentation to take the next step.
- Reduce time-to-first-course: prioritize setup, authoring, preview, publish, and invite flows.
- Build trust around AI: label AI output, show citations, expose uncertainty, and design graceful refusal states.
- Make tenant boundaries invisible but safe: users should feel they are in their own workspace without seeing security mechanics.
- Keep admin UI dense, clear, and operational.
- Keep learner UI calm, focused, and low-friction.
- Prefer progressive disclosure over large setup screens.
- Always show save, publish, progress, and completion status clearly.
- Never expose cross-tenant data or cross-tenant existence.
- Avoid hard-coded branding assumptions.
- Design responsive web and PWA-friendly experiences, not native mobile apps.

---

6. Information Architecture

Admin Surface

Suggested primary navigation:

- Dashboard
- Courses
- Learners / Users
- Reports
- Branding
- Billing
- Settings

The admin surface should feel like an operational workspace. It should support scanning, repeated actions, and quick movement between setup, authoring, users, reporting, and billing.

Learner Surface

Suggested primary navigation:

- My Courses
- Course Player
- Progress / Certificates
- AI Tutor inside course context
- Profile / Account

The learner surface should minimize distractions and make the next learning action obvious.

Role-Based Visibility

Tenant Admin sees organization management, branding, users, billing, reports, and course areas. Course Owner sees course-authoring areas without tenant branding, billing, or user-management controls. Learner sees only learner-facing areas. Frontend visibility is not authorization; backend permissions remain authoritative.

---

7. Core User Journeys

Journey A: Self-Serve Tenant Signup

1. User lands on signup.
2. Creates account.
3. Creates tenant/workspace.
4. Chooses tenant subdomain.
5. Lands in onboarding wizard.
6. Adds basic branding.
7. Creates first course or starts from blank.
8. Invites first learner.

Required UI states:

- Loading state for account creation, tenant creation, subdomain checks, and invite submission.
- Inline validation for email, password, tenant name, and subdomain.
- Reserved subdomain error for names such as `www`, `app`, `api`, `admin`, `status`, `docs`, `blog`, and `mail`.
- Unavailable subdomain error.
- Success confirmation after workspace creation.
- Empty state for a new tenant with no courses or learners.
- Recoverable failure state with retry guidance.

Journey B: Admin Creates And Publishes A Course

1. Admin opens Courses.
2. Creates course draft.
3. Adds sections.
4. Adds lessons.
5. Adds text, video, image, and quiz blocks.
6. Reorders content.
7. Previews course.
8. Publishes course.
9. Sees audit/success confirmation.

UX requirements:

- Autosave or clear manual save state.
- Visible draft vs published status.
- Validation before publish.
- Clear unsaved changes warning.

- Preview mode.
- Empty lesson guidance.
- Media upload progress, success, and failure states.

Journey C: Learner Takes A Course

1. Learner logs in.
2. Sees enrolled courses.
3. Opens course.
4. Resumes last position.
5. Reads or watches lesson content.
6. Takes quiz.
7. Sees progress update.
8. Completes course.
9. Downloads certificate.

UX requirements:

- Progress bar.
- Resume CTA.
- Completion state.
- Quiz feedback.
- Accessible course-player layout.
- Mobile responsive design.
- Certificate download success and unavailable states.

Journey D: Learner Uses AI Tutor

1. Learner opens tutor panel inside course player.
2. Asks a course-related question.
3. Tutor streams an answer.
4. Answer includes citations/references to lesson sections.
5. Tutor refuses out-of-course questions.
6. Tutor refuses to give quiz answers verbatim where configured.

UX requirements:

- AI-generated label.
- Streaming state.
- Citation display.
- Uncertainty state.
- Out-of-scope refusal.
- Error and retry state.
- Friendly rate-limit, token-budget, or degraded-provider messaging.

Journey E: Admin Checks Progress And Reporting

1. Admin opens Reports.
2. Views course completion percentage.
3. Views per-learner progress.

4. Filters by course/status.
5. Exports CSV.

UX requirements:

- Useful empty state for new tenants.
- Simple filters.
- Export success and failure states.
- Loading skeleton.
- Permission error state.

Journey F: Billing And Plan Management

1. Admin opens Billing.
2. Sees current plan.
3. Sees usage versus plan limits.
4. Starts checkout or plan change.
5. Returns after Stripe checkout.
6. Sees updated subscription state.

UX requirements:

- Plan comparison.
- Usage limits.
- Upgrade/downgrade clarity.
- Trial state.
- Payment failure state.
- Important billing errors must not be hidden.

---

8. Screen Inventory

Screen	User role	Purpose	Primary actions	Key data displayed	Required states
Landing/pricing entry point	Prospect/Admin	Explain plans and enter signup or checkout	View plans, start signup	Plan names, prices, limits, trial rule	Loading, error, selected plan
Signup	Prospect	Create first user account	Submit account details	Email, password requirements	Validation, loading, success, failure
Login	Tenant Admin/Course Owner/Learner	Authenticate user	Login, request magic link	Email/password or magic link option	Validation, loading, error
Magic link requested	Tenant Admin/Course Owner/Learner	Confirm email sent	Resend, return to login	Destination email hint	Success, resend cooldown, failure

Screen	User role	Purpose	Primary actions	Key data displayed	Required states
Tenant creation	Admin	Create workspace	Enter tenant name/subdomain	Workspace name, subdomain	Reserved subdomain, unavailable subdomain, loading, success
Onboarding wizard	Admin	Guide first setup	Add branding, create first course, invite learner	Progress through setup steps	Empty, skipped, completed, failure
Admin dashboard	Admin	Show tenant value quickly	View courses/reports, export CSV	Completion, enrolments, active courses	Loading, empty, error
Course list	Admin	Manage courses	Create, edit, publish/unpublish	Course title, status, updated date, enrolments	Loading, empty, filters, permission error
Course create/edit	Tenant Admin/Course Owner	Edit course metadata	Save, preview, publish	Title, description, status, visibility	Validation, unsaved, saved, publish-blocked
Course builder	Tenant Admin/Course Owner	Structure course content	Add/reorder sections and lessons	Sections, lessons, status	Empty, drag state, saving, error
Lesson editor	Tenant Admin/Course Owner	Build lesson blocks	Add text/video/image/quiz blocks	Blocks, media, quiz options	Empty lesson, upload, validation, unsaved
Media upload modal/page	Tenant Admin/Course Owner	Upload lesson media	Select, upload, retry, remove	File name, type, size, progress	Uploading, success, failure
Course preview	Tenant Admin/Course Owner	Review learner view before publish	Navigate lessons, return to edit, publish	Course content as learner sees it	Loading, missing content, validation warning
Learner course catalog	Learner	Show assigned/enrolled courses	Open, resume	Course card, progress, due/status if present	Empty, loading, error
Course player	Learner	Consume lesson content	Resume, mark complete, navigate lessons	Lesson content, progress, next lesson	Loading, empty, completed, error
Quiz screen	Learner	Answer quiz questions	Select answer, submit	Question, options, feedback	Unanswered, submitted, passed, failed
Completion/certificate screen	Learner	Confirm completion and obtain proof	Download certificate	Completion date, certificate info	Certificate unavailable, generating, ready, failure

Screen	User role	Purpose	Primary actions	Key data displayed	Required states
AI tutor panel	Learner	Ask course-grounded questions	Ask, retry, open citations	Messages, citations, AI label	Streaming, citation, refusal, uncertainty, rate limit, error
Learner progress report	Admin	Review learner-level progress	Filter, inspect learner	Learner, course, completion %, status	Loading, empty, permission error
CSV export state	Admin	Export reports	Download or retry	Export format/status	Preparing, success, failure
Branding settings	Admin	Configure tenant look	Upload logo, set primary colour, save	Logo, primary colour, subdomain	Preview, validation, saved, failure
Billing/plan page	Admin	Manage plan and subscription	Checkout, change plan, manage billing	Current plan, usage, limits, trial, payment status	Trial, active, past due, cancelled, checkout return
Account/profile	Admin/Learner	Manage personal settings	Update profile, logout	Name, email, role	Saving, success, validation, error
404 error page	All	Hide missing or inaccessible resources	Return home	Safe error message	Not found; no cross-tenant existence leak
403 error page	All	Explain insufficient permission	Return to allowed area	Safe permission message	Permission denied
500 error page	All	Recover from unexpected failure	Retry, contact support	Request ID	Error, retry
Loading and empty states	All	Cover async and new-tenant states	Continue, create first item, retry	Context-specific guidance	Skeleton, empty, recoverable failure

9. Component Requirements

- App shell/sidebar/topbar.
- Tenant branding header or workspace indicator, if applicable.
- Course card.
- Course status badge.
- Progress bar.
- Lesson navigation sidebar.
- Content block editor.
- Quiz question component.
- AI tutor chat panel.
- Citation chip/card.
- Empty state component.
- Error state component.

- Loading skeletons.
- Toast/notification system.
- Modal/dialog.
- Data table.
- Filter bar.
- CSV export button/state.
- Plan/pricing card.
- Usage limit meter.
- Form field/error pattern.
- Audit/success confirmation pattern.

---

10. Design System Direction

The visual direction should be professional SaaS: clean, modern, trustworthy, and efficient. Admin screens can be information-dense, but they should remain easy to scan. Learner screens should be calmer, more focused, and distraction-free.

The design system must support tenant branding through tokens:

- Logo.
- Primary colour.
- Accent colour.
- Locale.
- Time zone.
- Email template branding in a later phase.

Technical fit matters. Designs should work naturally with TailwindCSS and shadcn/ui. Avoid custom CSS-heavy one-off patterns unless they are justified by a core experience need.

Accessibility baseline:

- WCAG 2.1 AA contrast.
- Responsive web first.
- Keyboard navigation.
- Clear focus states.
- Readable typography.
- Components that support loading, disabled, error, success, empty, and permission states.

---

11. Data And Privacy UX Requirements

- Tenant data isolation must be reflected through safe, generic errors. Do not reveal whether a resource exists in another tenant.
- Missing, unauthorized, cross-tenant, and deleted resources should avoid existence-confirming language.
- UI errors must not expose sensitive data, credentials, tokens, private course content, billing details, or audit internals.
- AI responses are advisory and must be visibly AI-generated.
- Tutor output should cite course content where possible and state uncertainty when grounding is weak.
- Learner PII should be minimized in screens, exports, and error states.
- GDPR export/deletion flows should be designed where surfaced in MVP or near-term admin settings.

- Audit-sensitive actions need confirmation, clear result messaging, and safe failure recovery. This includes billing, enrolment, completion, certificates, publishing, RBAC, and admin actions.

12. Non-Functional UX Requirements

- Responsive layouts for mobile, tablet, and desktop.
- Fast perceived performance with skeletons or meaningful pending states.
- Loading states for all async operations.
- Empty states for all newly created tenant areas.
- Form validation before submit.
- Accessible keyboard navigation.
- Sensible focus states.
- Readable typography across viewport sizes.
- Clear error recovery.
- Progressive onboarding that lets users reach first value quickly.
- No text overlap, layout jumping, or hidden primary actions on small screens.

13. MVP Acceptance Criteria For Design

- A new tenant can understand what to do next after signup.
- Admin can create and publish a simple course without documentation.
- Learner can resume and complete a course on mobile and desktop.
- AI tutor is discoverable inside course context.
- AI tutor citations, refusals, uncertainty states, streaming states, and error states are designed.
- Admin can view progress and export CSV.
- Branding can be configured and reflected in the app.
- Billing and plan state are understandable.
- Empty, loading, error, success, disabled, and permission states are covered.
- Designs do not include MVP-excluded features as if they are available now.

14. Open Questions For Product/Design

- Should the MVP include a marketing landing page, or only signup/pricing entry?
- How much onboarding wizard depth is acceptable before first course creation?
- Should course creation start from a template or blank course only?
- What certificate layout is required for MVP?
- How prominent should AI tutor be in the course player?
- Should admin and learner experiences share the same shell or use separate layouts?
- What level of tenant branding should be visible in MVP emails?
- Should CSV export be a simple download action or a background job with notification?
- What plan-limit warnings should be shown before hard blocking?
- Should the admin dashboard prioritize setup progress, learning analytics, or billing/usage health for a brand-new tenant?
- Which course content types need first-class design polish for the earliest design partner: text, video, image, or quiz?

15. Deliverables Expected From UI/UX Designer

- User flow diagrams.
- Information architecture/site map.
- Low-fidelity wireframes.
- High-fidelity Figma designs.
- Clickable prototype for core flows.
- Responsive variants for mobile, tablet, and desktop.
- Component library/design system foundations.
- Empty, loading, error, success, disabled, and permission states.
- Handoff notes for frontend implementation.

Assumptions

- The current MVP implementation is functionally complete, but the next frontend pass is expected to refine the product experience and visual system.
- MVP uses custom subdomains, not custom domains.
- Native mobile apps are out of scope; responsive web and PWA-compatible patterns are in scope.
- AI course generation, voice tutor, forums, marketplace, SSO/MFA, SCORM/xAPI, and MCP/agentic tools are future-phase items.
- Pricing and plan limits are product hypotheses and should be treated as configurable product data, not permanent hard-coded design assumptions.